Django Task Workers in Subinterpreters

melhin

Intro

- I am Melhin Ahammad
- Been working as an engineer for quite some time now
- Mostly backend dev and do things in infra
- I work at Kraken technologies like some other speakers here

Before we start

Antony Shaw's PyCon talk on subinterpreters inspired me to explore this.

- An experiment to combine two new features
- Exploring how new Python primitives compose with familiar Django patterns

Requirements

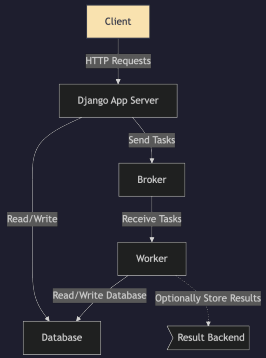
- Python 3.14+ (stable release October 2025)
- Django 6.0+ (for Django Tasks framework)

Why ?

Deferred Processing in Django Applications

- Avoid blocking main request/response cycle
- Not all applications have a linear execution path
- You might need to offload CPU intensive work

How does an deferred task processing architecture look like?



Lets look at the components

Django server deployment

- gunicorn
- daphne
- uvicorn
- hypercorn

Task processing frameworks (Before Django 6)?

- Celery
- RQ
- Django-Q

The Cost of Running a Separate Worker

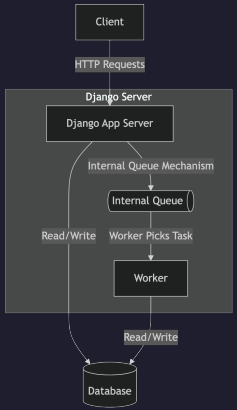
- Broker + worker + web server — all deployed and monitored separately
- More moving parts than small/medium apps often need
- Every extra process is another thing to configure and reason about

The Real Why ?

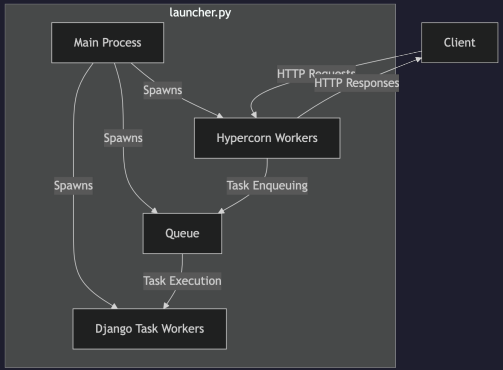
Fediverse (jointhefediverse.net)

- A collection of federated social media platforms (Mastodon, Pleroma, etc.)
- I really wanted to run my own fediverse server without any hassle

I want my server to look like this ?



Here's what a subinterpreter-based architecture might look like

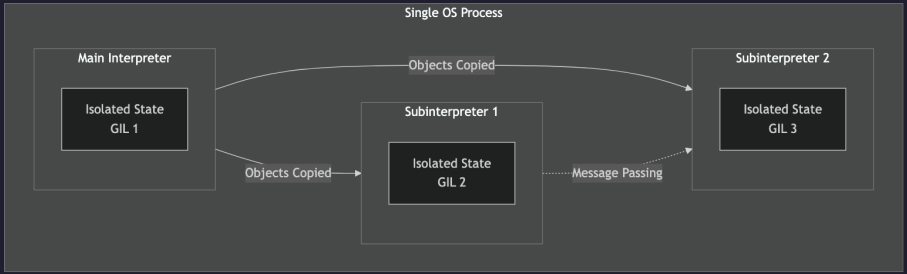


Let's look at each component

What are Subinterpreters?

- Per GIL interpreter introduced in Python 3.12 (PEP 684)
- Multiple Interpreters in the Stdlib Python 3.14
- Enable multiple independent Python interpreters in a single process
- Single OS process (lower overhead than multiprocessing) with isolated Python objects

Subinterpreter Architecture



How Isolation Works In Subinterpreters

- Isolated state: Separate imports, builtins, and variable namespaces per interpreter
- Objects are copied: Most passed objects pickled (copied); prevents shared mutable state bugs
- Message passing: Primary communication; a few types like `Queue` actually shared

Why Not Threads, Asyncio or Multiprocessing?

Feature	🧵 Threads	⚡ Asyncio	🔀 Multiprocessing	🐍 Subinterpreters
Same process	✅	✅	❌	✅
True parallelism	❌ GIL	❌ single thread	✅	✅ own GIL
Isolation	❌ shared	❌ shared	✅	✅
No IPC / broker	✅	✅	❌ pipes	✅ in-memory
One process	✅	✅	❌	✅

No IPC Layer

Subinterpreters keep the queue inside the process:

- No OS pipes, no broker
- Shared memory
- One PID, one thing to deploy and monitor

A simple example of using InterpreterPoolExecutor in Python 3.14

```python
import concurrent.futures
from concurrent.futures.interpreter import InterpreterPoolExecutor

def calc_fib(n):
    ...

with InterpreterPoolExecutor(max_workers=5) as executor:
    futures = {executor.submit(calc_fib, n): n for n in range(10)}
    for future in concurrent.futures.as_completed(futures):
        n = futures[future]
        data = future.result()
        print(f"fib({n}): {data}")
```

Django Tasks

- Introduced in Django 6.0
- Provides a unified way to define and run background tasks
- Abstracts away implementation details
- Allows switching between different backends

Interchangeability of Backends

- Same Task, Different Backends: A task defined with `@task` can run on any backend without modifying its code. For example:
  - Run locally with `ImmediateBackend` for testing.
  - Deploy with `SubinterpreterBackend` for in-process execution.
  - Switch to `DatabaseBackend` for distributed task processing.
- Hybrid Approaches: Combine multiple backends for different use cases (e.g., in-process for low-latency tasks, broker-based for long-running tasks).

Configuring tasks to run in Subinterpreter

```python
from django_tasks.backends import BaseBackend

class InterpreterQueueBackend(BaseBackend):
    def enqueue(self, task: Task, ...) -> None:
        self.validate_task(task)
        worker_queue = get_shareable_queue("worker_queue")
        worker_queue.put(shareable_task)
        return result

# Configure Django Tasks to use our backend
TASKS = {
    "default":
        {"BACKEND":"parimitham.core.interpreter_queue_backend.InterpreterQueueBackend"}
}
```

Configuring tasks to run in Subinterpreter (2)

```
1  @task
2  def process_cpu_intensive_task():
3      pass
4
5  @require_http_methods(["GET"])
6  def delayed_hello(request):
7      cpu_intensive_work_task.enqueue()
8      return JsonResponse({"message": "Task enqueued"})
9
10
```

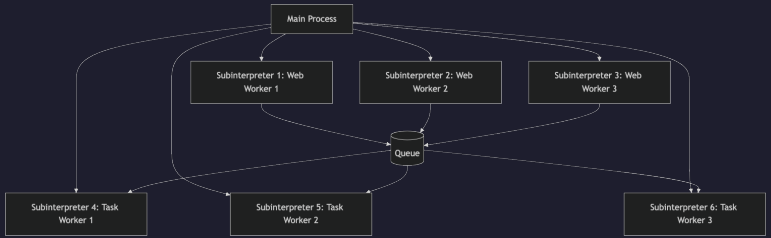
SubInterpreter Task Worker

```python
class InterpreterWorker:
    def __init__(
        self,
        *,
        queue_names: list[str],
        interval: float,
        batch: bool,
        backend_name: str,
        startup_delay: bool,
        max_tasks: int | None,
        worker_id: str,
    ):
        ...
```

SubInterpreter Task Worker

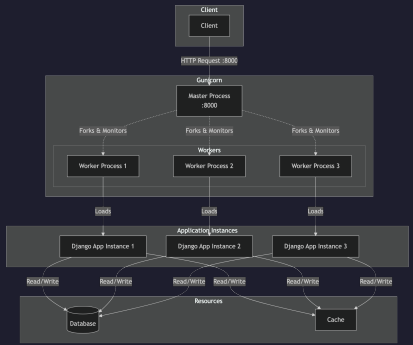
```python
def run(self, timeout: float = 1.0) -> None:
    ...
    worker_queue = get_shareable_queue("worker_queue")
    ...

    while self.running:
        try:
            shareable_task = worker_queue.get(timeout=timeout)
            logger.info("Got task from queue: %s", shareable_task)
            # An import string based logic to pick up tasks and run them
            self.run_task(shareable_task)

        except QueueEmpty:
            continue
        except (OSError, RuntimeError):
            time.sleep(self.interval)
```

When we combine these two together we get something like this

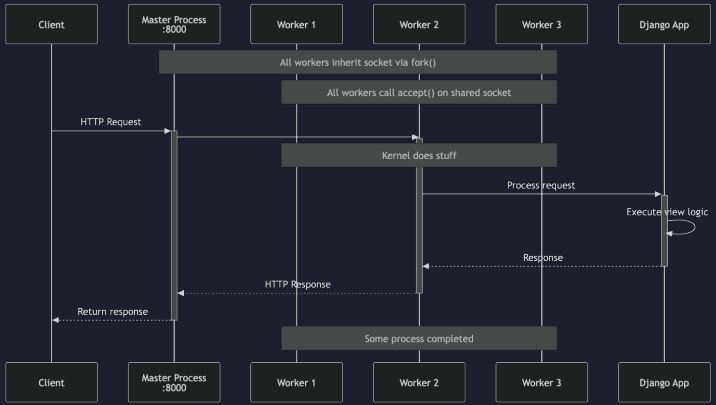


How we run production webservers ?

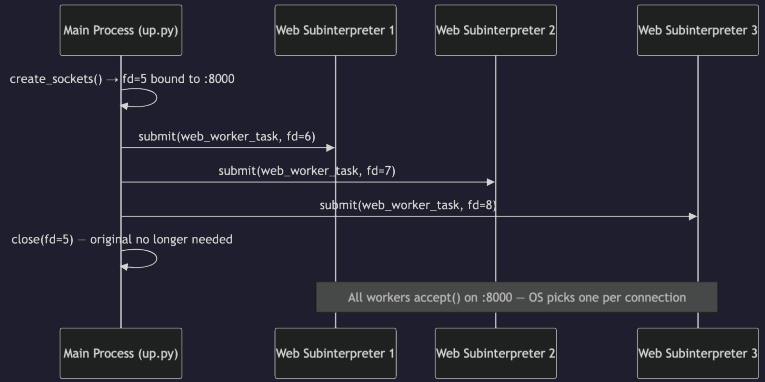
To understand why subinterpreters help, let's see how production servers work today.



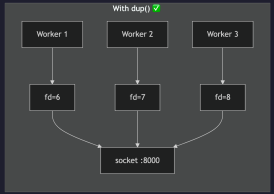
How Gunicorn Routes Requests

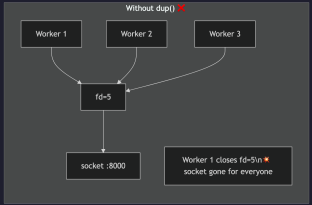


Serving HTTP from Subinterpreters



Socket Duplication: Why dup()?





Demo !

Running dependent workflows

```
@task()
def store_self_post(activity):
    # Store a self-authored ActivityPub post and broadcast to followers.
    ...
    broadcast_to_followers.enqueue(activity)


@task()
def broadcast_to_followers(activity):
    # Broadcast an ActivityPub post to all followers.
    ...
    for follower in followers:
        send_activity_to_follower.enqueue(follower.id, activity)


@task()
def send_activity_to_follower(follower_id, activity):
```

Error Handling & Graceful Shutdown

- Failures are isolated: Subinterpreter crashes don't take down the whole system
- Startup sequencing matters: Ensure pool is initialized before enqueuing tasks
- Queue as coordination mechanism:
  - Signal workers to finish current tasks
  - Clean up subinterpreters and queues properly

Scaling Beyond Single Server

Single Server (Subinterpreters)

- ✅ Low overhead, no broker
- ✅ Tasks in-memory, fast
- ❌ Queue lost on restart
- ❌ Single server only

Multiple Servers (Database Backend)

- ✅ Tasks persisted in DB
- ✅ Multiple servers share queue

```
TASKS = {
    "default": { "BACKEND": "
django.tasks.backends.database.D
atabaseBackend"
    }
}
```

Subinterpreter Challenges and Limitations

- Technology Maturity
- Key Limitations:
  - Object sharing is evolving.
  - Serialization/deserialization for task execution.
  - Debugging and monitoring tools.
- Performance Unknowns:
  - Overhead
  - Memory usage

Who Is This For Today?

- Django + DRF + Django templates
- PostgreSQL
- Any pure Python dependencies

Blocked if you use

- numpy — issue #24755 (https://github.com/numpy/numpy/issues/24755)
- PyO3-based packages (ruff, cryptography, tokenizers) — issue #3451 (https://github.com/PyO3/pyo3/issues/3451)
- pydantic — issue #12452 (https://github.com/pydantic/pydantic/issues/12452)
- The blocker: C extensions that haven't implemented multi-phase initialisation (PEP 489)

Closing Thoughts

- Explore Django Tasks today
- Subinterpreters unlock a new form of parallelism for deferred workflows
- Pure Python + psycopg? Give it a try now
- Watch the ecosystem

Thank you!

Antony Shaw's Blog

github.com/melhin/parimitham

Python Forum Discussion

Jamie's Blog